Memory Management Aspects
`mmap()` and `madvise()`
A Linux Systems Programming Tutorial

Debarshi Roy[1] Sayan Das[1]

[1]CSE231 - Operating Systems Laboratory
Indraprastha Institute of Information Technology Delhi

OS Lab Tutorial Session
4 November 2026

Tutorial Roadmap

1. Understand virtual memory, pages, and memory mappings.
2. Learn the purpose and arguments of `mmap()`.
3. Create file-backed and anonymous mappings in C.
4. Learn how `madvise()` communicates expected access patterns.
5. Compile, run, inspect, and trace the example programs.

By the end of this tutorial

You should be able to create and release a mapping, distinguish shared from private mappings, choose a suitable memory-advice value, and inspect a process's mapped regions.

What We Will Practise

Concepts

- Process virtual address space
- Pages and page faults
- File-backed vs. anonymous memory
- Shared vs. private mappings
- Kernel memory-use advice

Commands and outcomes

- `man 2 mmap`, `man 2 madvise`
- `getconf PAGESIZE`
- `gcc` and program output
- `/proc/PID/maps`
- `strace` system-call trace

Important

`mmap()` and `madvise()` are C library interfaces to kernel services, not standalone shell commands. We call them from a program and use shell tools to inspect their effects.

A Few Terms Before We Start

Virtual address An address used by a process; the OS and hardware translate it to a physical location.

Page A fixed-size unit used by the virtual-memory system.

Mapping A relationship between a process's virtual address range and a backing object or anonymous memory.

Page fault An event raised when an access needs the OS to establish or validate a page mapping.

Check your machine

Run `getconf PAGESIZE`. Do not assume that every Linux machine uses the same page size.

Section Overview

Memory Mapping Fundamentals

mmap(): Creating a Mapping

madvise(): Advising the Kernel

Hands-on Lab, Debugging, and Recap

1 Memory Mapping Fundamentals

2 mmap(): Creating a Mapping

3 madvise(): Advising the Kernel

4 Hands-on Lab, Debugging, and Recap

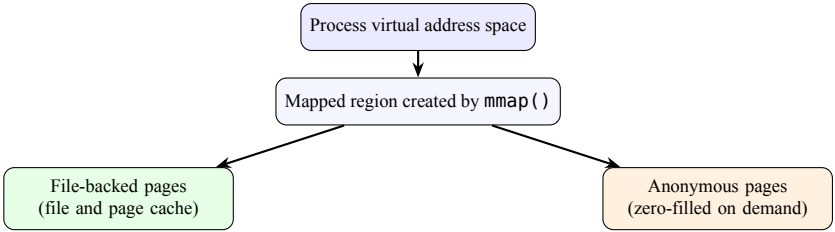
Why Do We Need Memory Mapping?

- A process works with a virtual address space; it does not directly choose physical RAM locations.
- The OS maintains mappings between virtual pages and physical page frames or backing storage.
- `mmap()` maps a file, device, or anonymous memory region into a process's virtual address space.
- The program accesses that region using ordinary pointer reads and writes, subject to the mapping's permissions.

Typical use cases

Large-file access, memory-mapped databases, shared-memory communication, loading executable files and libraries, and large anonymous buffers.

A Simple Virtual-Memory View



Key idea

A mapping establishes a virtual address range. Physical pages are commonly loaded or allocated as the process accesses the range.

File-Backed and Anonymous Mappings

File-backed mapping

- Maps bytes from an open file descriptor.
- Useful for large files and structured data.
- `MAP_SHARED`: writes can be visible to other mappers and are carried through to the file.
- `MAP_PRIVATE`: writes use copy-on-write and remain private.

Anonymous mapping

- Has no ordinary file backing.
- Commonly used for process buffers and allocations.
- Initially reads as zero-filled memory.
- Typical flags: `MAP_PRIVATE | MAP_ANONYMOUS`.

Remember

Creating a mapping does not mean every page is already resident in RAM.

Section Overview

Memory Mapping Fundamentals

mmap(): Creating a Mapping

madvise(): Advising the Kernel

Hands-on Lab, Debugging, and Recap

1 Memory Mapping Fundamentals

2 mmap(): Creating a Mapping

3 madvise(): Advising the Kernel

4 Hands-on Lab, Debugging, and Recap

The `mmap()` Function

Header and function signature

```
#include <sys/mman.h>

void *mmap(void *addr, size_t length, int prot,
           int flags, int fd, off_t offset);
```

Argument	Meaning
`addr`	Suggested start address; normally pass `NULL` and let the kernel choose.
`length`	Number of bytes to map; must be greater than zero.
`prot`	Access permissions, such as read, write, execute, or none.
`flags`	Mapping type and additional options.
`fd`	Open file descriptor; use `-1` for an anonymous mapping.
`offset`	File offset; must be page-aligned. For an anonymous mapping, use zero.

Protection and Mapping Flags

Value	Meaning
`PROT_READ`	Pages may be read.
`PROT_WRITE`	Pages may be written.
`PROT_EXEC`	Pages may be executed.
`PROT_NONE`	Pages may not be accessed.
`MAP_SHARED`	Updates are shared with other mappings; for a file-backed mapping, updates are carried through to the file.
`MAP_PRIVATE`	Private copy-on-write mapping; writes are not propagated to the file or other mappings.
`MAP_ANONYMOUS`	No ordinary file backing; the initial contents are zero-filled.

Flag selection

Choose one of `MAP_SHARED` or `MAP_PRIVATE`. Add `MAP_ANONYMOUS` when no file is being mapped.

First Commands: Explore the System

```
# Read the system-call documentation
man 2 mmap
man 2 madvise

# Find the page size used by this machine
getconf PAGESIZE

# Inspect the current shell's memory map
cat /proc/$$/maps | less
```

What should you observe?

- The manual pages describe the arguments, flags, errors, and advice values.
- `getconf PAGESIZE` prints the page size in bytes.
- The final command displays the mapped regions of the current shell process.

How to Read `/proc/PID/maps`

A typical line has this structure:

```
address-range permissions offset device inode pathname
```

- `address-range`: start and end virtual addresses.
- `permissions`: `r` = read, `w` = write, `x` = execute.
- `p` = private mapping; `s` = shared mapping.
- `offset`: offset in the backing file, when applicable.
- `pathname`: file name for file-backed mappings; anonymous regions may have no ordinary file path.

Try it

In the output of `cat /proc/$$/maps`, find a line with `rw-p` and another line with an executable permission.

Example 1: Open and Size a File

```c
int fd = open("mapped_demo.txt",
              O_RDWR | O_CREAT | O_TRUNC, 0644);
if (fd == -1) {
    perror("open");
    return EXIT_FAILURE;
}

if (ftruncate(fd, map_length) == -1) {
    perror("ftruncate");
    close(fd);
    return EXIT_FAILURE;
}
```

What happened?

The program opens the file for reading and writing, creates it if needed, truncates old contents, and sets its size to `map_length` bytes before mapping it.

Example 1: Create the Mapping

```c
char *region = mmap(NULL, map_length,
                    PROT_READ | PROT_WRITE,
                    MAP_SHARED, fd, 0);

if (region == MAP_FAILED) {
    perror("mmap");
    close(fd);
    return EXIT_FAILURE;
}
```

Important checks

- `mmap()` returns `MAP_FAILED` on error; do not check for `NULL`.
- The mapping is readable and writable because of `PROT_READ | PROT_WRITE`.
- `MAP_SHARED` makes updates visible to other mappers and carries file-backed updates through to the file.

Example 1: Write and Synchronize

```c
const char *message = "Hello from mmap()!\n";

int written = snprintf(region, map_length, "%s", message);
if (written < 0 || (size_t) written >= map_length) {
    fprintf(stderr, "snprintf failed or message too long\n");
    munmap(region, map_length);
    close(fd);
    return EXIT_FAILURE;
}

if (msync(region, map_length, MS_SYNC) == -1)
    perror("msync");

printf("Mapped memory says: %s", region);
```

Expected outcome

The program prints `Hello from mmap()!`. `msync(MS_SYNC)` requests synchronous write-back of the mapped file range.

Example 1: Release the Resources

```c
if (munmap(region, map_length) == -1) {
    perror("munmap");
    close(fd);
    return EXIT_FAILURE;
}

if (close(fd) == -1) {
    perror("close");
    return EXIT_FAILURE;
}
```

Remember

`munmap()` removes the mapped range from the process's virtual address space. Closing the file descriptor is a separate operation.

Compile, Run, and Check the File

```
# Compile with warnings enabled
gcc -std=c11 -Wall -Wextra -O0 mmap_file.c -o mmap_file

# Execute the program
./mmap_file

# Display the message stored through the mapping
head -c 19 mapped_demo.txt
printf '\n'
```

Expected output

```
Mapped memory says: Hello from mmap()!
Hello from mmap()!
```

The first line is printed by the C program; the second is read back from the file.

What Changes with `MAP_PRIVATE`?

`MAP_SHARED`

- Writes can be seen by other processes mapping the same region.
- File-backed updates are carried through to the file.
- Use `msync()` when explicit synchronization is required.

`MAP_PRIVATE`

- Writes create private copy-on-write pages.
- Other mappers do not see those writes.
- The writes are not carried through to the underlying file.

Quick experiment

Change `MAP_SHARED` to `MAP_PRIVATE`, run the program, and inspect the file. The private writes should not change the file's contents.

Mapping Lifecycle

1. Open the file with access permissions compatible with the mapping.
2. Make the file large enough for the range you will access.
3. Call `mmap()` and check for `MAP_FAILED`.
4. Access only addresses within the valid mapped range.
5. Call `msync()` when explicit synchronization is required for a shared file mapping.
6. Call `munmap()` when the mapping is no longer needed; close the file descriptor separately.

Common `mmap()` Pitfalls

- **Wrong error check:** test against `MAP_FAILED`, not `NULL`.
- **Bad offset:** the file offset passed to `mmap()` must be page-aligned.
- **File too short:** accessing a mapped file page beyond the current end of the file can raise `SIGBUS`.
- **Wrong permissions:** the file descriptor's open mode must be compatible with the requested mapping and protections.
- **Resource leak:** release the mapping with `munmap()` when finished.

Useful distinction

The mapping length may be any positive byte count; the file offset, not the length, is the argument that must be page-aligned on Linux.

Memory Mapping Fundamentals

mmap(): Creating a Mapping

madvise(): Advising the Kernel

Hands-on Lab, Debugging, and Recap

Section Overview

1 Memory Mapping Fundamentals

2 mmap(): Creating a Mapping

3 madvise(): Advising the Kernel

4 Hands-on Lab, Debugging, and Recap

What Does `madvise()` Do?

Function signature

```c
#include <sys/mman.h>

int madvise(void *addr, size_t length, int advice);
```

- It tells the kernel how the application expects to use an existing memory range.
- The kernel may adjust read-ahead, reclaim, or page-handling decisions.
- Most advice values are hints, not guarantees of performance.
- On Linux, `addr` must be page-aligned; the specified length need not be page-aligned.

Difference in one line

`mmap()` creates a mapping; `madvise()` gives the kernel usage advice about an existing range.

Common `madvise()` Advice Values

Advice	Application's message to the kernel
`MADV_NORMAL`	Use the default behavior.
`MADV_RANDOM`	Expect page references in random order; read-ahead may be less useful.
`MADV_SEQUENTIAL`	Expect sequential access; the kernel may read ahead more aggressively and reclaim pages after use.
`MADV_WILLNEED`	Expect access soon; the kernel may read some pages ahead.
`MADV_DONTNEED`	The application is finished with the range for now; the kernel may release associated resources.

Interpretation

`MADV_RANDOM`, `MADV_SEQUENTIAL`, and `MADV_WILLNEED` describe expected access patterns. They do not change the program's logical data values by themselves.

Choosing an Advice Value: Code Examples

```c
/* 'map' is an existing mapping of 'length' bytes. */

/* A scan from the beginning to the end of a mapped file: */
if (madvise(map, length, MADV_SEQUENTIAL) == -1)
    perror("madvise(MADV_SEQUENTIAL)");

/* A range likely to be accessed soon: */
if (madvise(map, length, MADV_WILLNEED) == -1)
    perror("madvise(MADV_WILLNEED)");

/* A workload that jumps between unrelated pages: */
if (madvise(map, length, MADV_RANDOM) == -1)
    perror("madvise(MADV_RANDOM)");
```

Expected outcome

Return value `0` means the advice call succeeded. It does not guarantee prefetching or a speedup; benchmark the actual workload.

Understanding `MADV_DONTNEED`

- Use it when the application no longer needs the current contents of a mapped range.
- It allows the kernel to discard or reclaim pages associated with that range.
- What a later read observes depends on the mapping type.

Linux: later access after successful `MADV_DONTNEED`

- **Private anonymous mapping:** pages are supplied again as zero-filled-on-demand pages.
- **Shared file or shared anonymous mapping:** contents are repopulated from the up-to-date backing object; the pages may not be physically freed immediately.

Important

`MADV_DONTNEED` does not mean ”erase the file”. Use it only when the application can safely give up the current contents of the range.

Example 2: Create an Anonymous Mapping

```c
long page_result = sysconf(_SC_PAGESIZE);
if (page_result <= 0) {
    perror("sysconf");
    return EXIT_FAILURE;
}

size_t page_size = (size_t) page_result;
size_t length = 4 * page_size;

char *mem = mmap(NULL, length, PROT_READ | PROT_WRITE,
                 MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
if (mem == MAP_FAILED) {
    perror("mmap");
    return EXIT_FAILURE;
}
```

What happened?

The program creates a private anonymous region spanning four system pages. The initial contents are zero-filled on demand.

Example 2: Write, Advise, and Read Again

```c
for (size_t i = 0; i < 4; ++i)
    mem[i * page_size] = (char) ('A' + i);

printf("Before: %c %c %c %c\n",
       mem[0], mem[page_size],
       mem[2 * page_size], mem[3 * page_size]);

if (madvise(mem, length, MADV_DONTNEED) == -1) {
    perror("madvise");
    munmap(mem, length);
    return EXIT_FAILURE;
}
```

What happens next?

The program reads the same locations again. For this Linux private-anonymous mapping, those reads fault in zero-filled pages. The complete program is supplied as `madvise_demo.c`.

Example 2: Expected Output and Cleanup

```c
printf("After : %d %d %d %d\n",
       (unsigned char) mem[0],
       (unsigned char) mem[page_size],
       (unsigned char) mem[2 * page_size],
       (unsigned char) mem[3 * page_size]);

if (munmap(mem, length) == -1) {
    perror("munmap");
    return EXIT_FAILURE;
}
```

Expected output on Linux

```
Before: A B C D
After : 0 0 0 0
```

This output applies to the private anonymous mapping used in this example after a successful `MADV_DONTNEED` call.

Memory Mapping Fundamentals
mmap(): Creating a Mapping
madvise(): Advising the Kernel
Hands-on Lab, Debugging, and Recap

Compile and Run the `madvise()` Demo

```
# Compile the complete example
gcc -std=c11 -Wall -Wextra -O0 madvise_demo.c -o madvise_demo

# Run it
./madvise_demo
```

Verify the page size

```
getconf PAGESIZE
```

The C program obtains the page size at runtime using `sysconf(_SC_PAGESIZE)`, so it does not assume a fixed 4096-byte page.

Linux-Specific Advice: Know Before Using

- `MADV_HUGEPAGE`: mark a range as a candidate for Transparent Huge Pages.
- `MADV_NOHUGEPAGE`: advise against Transparent Huge Pages for a range.
- Other Linux-specific values support cases such as memory reclaim and fork behavior.

Portability and kernel versions

The available advice values and their exact behavior depend on the kernel version, configuration, and mapping type. Check `man 2 madvise` on the target machine before using a Linux-specific value.

Memory Mapping Fundamentals

mmap(): Creating a Mapping

madvise(): Advising the Kernel

Hands-on Lab, Debugging, and Recap

Section Overview

1 Memory Mapping Fundamentals

2 mmap(): Creating a Mapping

3 madvise(): Advising the Kernel

4 Hands-on Lab, Debugging, and Recap

Inspect a Process Mapping

```
# Inspect the current shell's mapping
cat /proc/$$/maps | less

# Optional summary, if procps is installed
pmap -x $$
```

What to look for

- Address ranges and file offsets.
- Permissions such as `rw-p` (read/write, private) and `rw-s` (read/write, shared).
- File pathnames for file-backed mappings.
- Anonymous regions without ordinary file pathnames.

Trace System Calls with `strace`

```
# Trace mapping-related calls as the program runs
strace -e trace=mmap,madvise,munmap ./madvise_demo

# Save the trace for later inspection
strace -o memory_trace.txt -e trace=mmap,madvise,munmap ./madvise_demo

grep -E 'mmap|madvise|munmap' memory_trace.txt
```

Expected observations

The trace shows calls such as `mmap(...)` returning an address, `madvise(...)` returning `0` on success, and `munmap(...)` releasing a region. Runtime libraries may also create mappings.

Permissions

Tracing a program started by `strace` usually does not require `sudo`. A lab server's security policy may still restrict tracing.

Common Errors: Mapping and File Access

Symptom	Check or correction
`mmap()` returns `MAP_FAILED`	Check `errno` with `perror()`; verify the file descriptor, protections, flags, length, and offset.
`SIGBUS` while accessing a mapped file	Ensure the accessed file offset is within the current file size; do not truncate the file while relying on pages beyond its new end.
Changes do not appear in the file	Check whether the mapping is `MAP_SHARED`; writes through `MAP_PRIVATE` are copy-on-write.
Mapping remains after use	Call `munmap(addr, length)` when finished.

Common Errors: Memory Advice

- **`madvise()` returns `-1`:** inspect `errno`; check that the address is page-aligned, the range is mapped, and the advice is supported.
- **Advice appears to do nothing:** most values are hints, not promises of a measurable performance improvement.
- **Unexpected data after `MADV_DONTNEED`:** check the mapping type; the follow-up behavior differs for private anonymous and shared mappings.
- **Tracing is denied:** check whether the server restricts `ptrace`; do not attempt to bypass the administrator's policy.

Guided Tutorial Exercise

1. Run the file-backed example with `MAP_SHARED`; change the string and verify the file.
2. Change the flag to `MAP_PRIVATE`; repeat the test and compare the file.
3. Create a `MAP_PRIVATE | MAP_ANONYMOUS` region and inspect its initial bytes.
4. Apply `MADV_SEQUENTIAL` to a sequential file scan.
5. Apply `MADV_DONTNEED` to the private anonymous mapping and inspect the next reads.
6. Use `strace` to identify the mapping, advice, and unmapping calls.

Expected learning outcomes

Explain which updates reach the file, what a later read observes after `MADV_DONTNEED`, and how Linux exposes a process's mapped regions.

Recap: Five Things to Remember

1. `mmap()` creates a region in a process's virtual address space.
2. `MAP_SHARED` shares updates; `MAP_PRIVATE` uses copy-on-write.
3. `MAP_ANONYMOUS` creates memory without ordinary file backing.
4. `madvise()` communicates expected usage; most advice values are hints.
5. `MADV_DONTNEED` has mapping-dependent follow-up behavior.

Final check

Can you explain the difference between mapping memory, advising the kernel, and unmapping memory?

References and Further Reading

- Linux man-pages: `mmap(2)`.
- Linux man-pages: `madvise(2)`.
- Linux man-pages: `msync(2)`.
- Linux man-pages: `proc_pid_maps(5)`.
- Linux man-pages: `strace(1)`.

Questions?